

1 - Pointer dan Array

IF230 – Algoritma & Struktur Data

Semester Genap 2021/2022

Outlines

Introduction

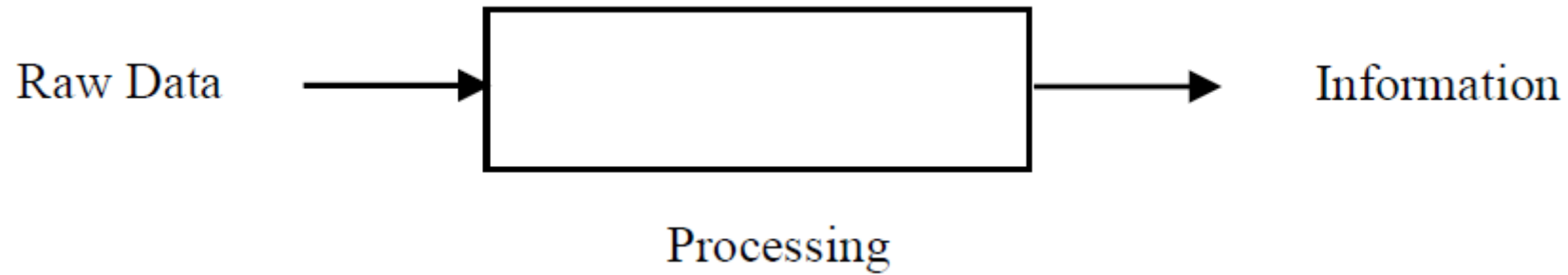
Array

Pointer

Exercises

Introduction

Data & Information

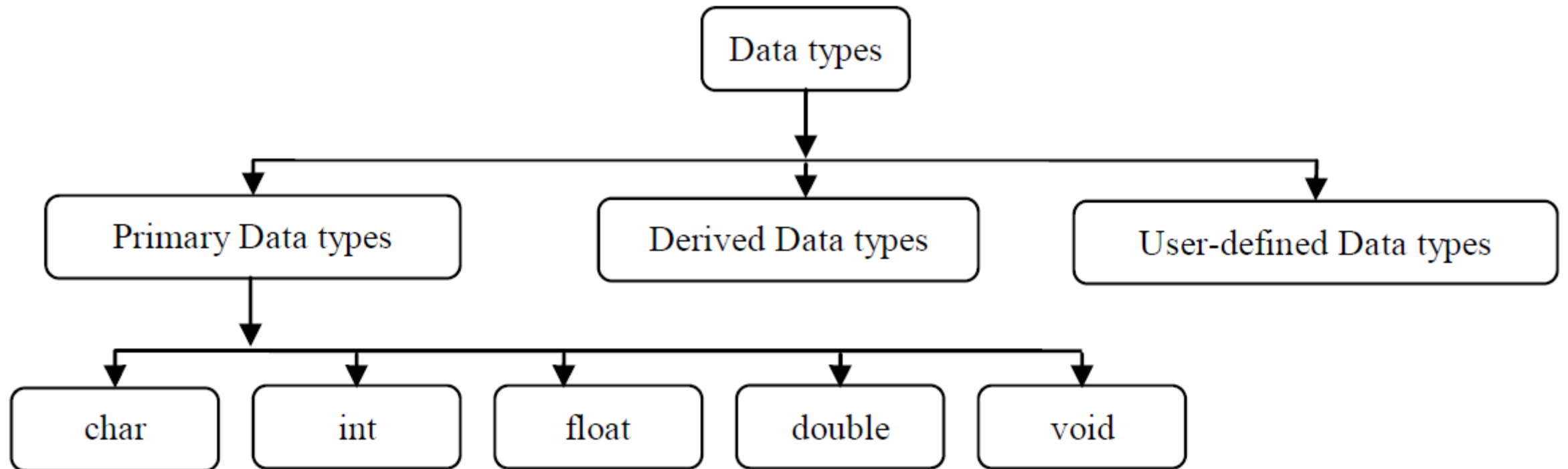


Definition

Data is the basic entity or fact that is used in a calculation or manipulation process.

The data represent quantities, characters, or symbols on which operations are performed by a computer, stored and recorded on either magnetic, optical, or mechanical recording media, and transmitted in the form of digital electrical signals.

Data Types in C



Data Structure

In Computer Science, **Data Structure** is defined as a mathematical or logical model of organizing the data items into computer memory in such a way that they can be used efficiently.

Data Structure refers to the study of data and representation of data objects within the program; i.e., the implementation of structured relationships among different data objects.

Example: lists, stacks, queues, heaps, search trees, hash tables, etc.

Program

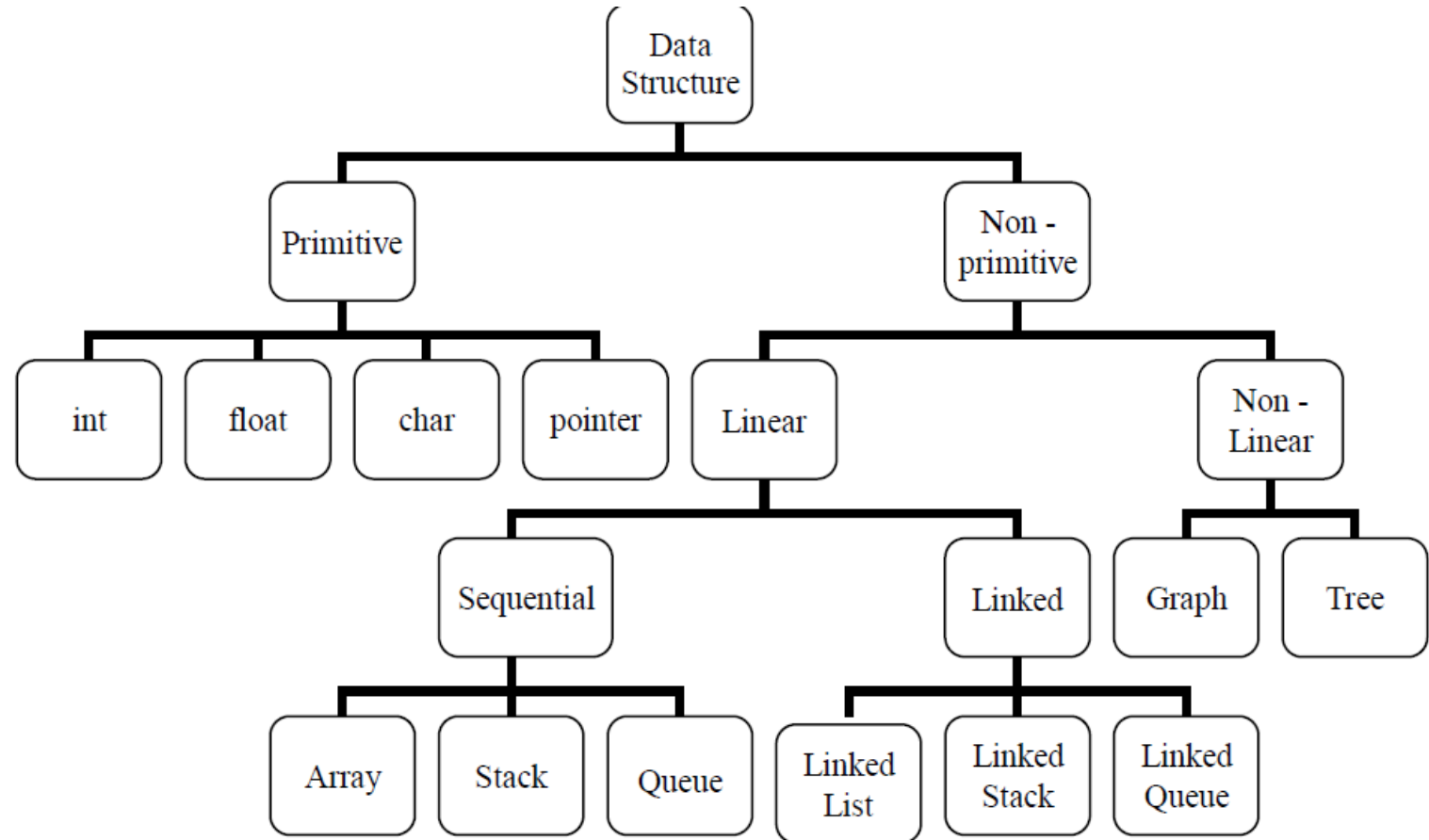
$$\textbf{Algorithm} + \textbf{Data Structure} = \textbf{Program}$$

A program is a set of instructions, which involve a computer performing some kind of computation or algorithm.

Data Structure affects the design of both structural and functional aspects of a program.

To implement a program of an algorithm, we should select an appropriate data structure for that algorithm. Therefore, the programs are inherited by an algorithm and its associated data structure.

Classification of Data Structure



Array

Array

Definition

An array is a collection of the same type of data items, which are stored in consecutive memory locations under a common name.

An array variable must be followed by square bracket enclosing size specification of the array.

The size signifies the number of elements of the array. That number should be a positive integer number (greater than 0).

One-dimensional Array

A one-dimensional array is one in which only one subscript is needed to specify a particular element of the array.

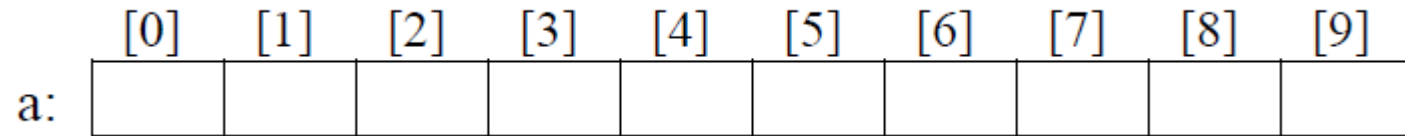
The one-dimensional array can be declared as follows:

```
data_type array_name [size];
```

where **array_name** is the name of an array and size is the number of elements of data type and **size** must be an integer constant specified within a pair of square brackets.

Example:

```
int a[10];
```



Array of size 10

Example

```
int i;  
for(i = 0; i < 10; i++) {  
    a[i] = 0;  
}
```

This loop sets 0 to all ten elements of the array a[0] to a[9] .

Example

```
int a[10];  
a = 0;           /* illegal */  
and  
int b[10];  
b = a;           /* illegal*/
```

```
int b[10];  
for(i = 0; i < 10; i++)  
    b[i] = a[i];
```

To **set** all of the elements of an array to some value, you must do so **one by one**.

To **copy** the contents of one array to another, you must again do so **one by one**.

Initialization of One-Dimensional Array

The syntax looks like this:

```
int a[10] = {0,1,2,3,4,5,6,7,8,9};
```

The list of values, enclosed in braces {}, separated by commas, provides the initial values for successive elements of the array.

If there are fewer initializers than elements in the array, the remaining elements are automatically initialized to 0.

The individual array elements are as follows:

a[0] = 0

a[1] = 1

a[2] = 2

a[3] = 3

a[4] = 4

a[5] = 5

a[6] = 6

a[7] = 7

a[8] = 8

a[9] = 9

Reading Array Elements

As an array may contain more than one element, so to read all the elements, a single call to the scanf

function will not serve our purpose.

Example:

If we have an array declared as

```
int a[10];
```

to read all the elements we can use the following statements:

```
for(i = 0; i < 10; i++)  
    scanf ("%d", &a[i]);
```

When the value of i is 0 then scanf() will read the value of a[0] element, if i is 1 it will read the value of a[1] and so on.

Writing Array Elements

Similarly, to print all the array elements of an array we can use the printf statement within a loop.

Considering the above array declaration `int a [10]`. We can use the following statements:

Example:

```
for(i = 0; i < 10; i++)  
    printf ("%d\t", a[i]);
```


Example Program

```
#include<stdio.h>

main()
{
    int a[10], max, min, i, n;
    printf("\nEnter the number of elements:");
    scanf ("%d",&n);
    printf("\nEnter %d elements:",n);
    for( i = 0 ; i < n; i++ )
        scanf("%d", &a[i]);

    /* Initialization */
    max = a[0];
    min = a[0];

    /* Finding Maximum and Minimum element */
    for(i = 0; i < n; i++) {
        if(a[i] > max) max = a[i];
        if(a[i] < min) min = a[i];
    }
    printf ("\n The maximum element is: %d", max);
    printf ("\n The minimum element is: %d", min);
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements: 3 2 9 5 4
The maximum element is: 9
The minimum element is: 2
```

Multi-Dimensional Arrays

Multi-dimensional array is nothing but Array of Arrays.

In fact, a two-dimensional array is an array of one-dimensional array. A three-dimensional array is an array of two-dimensional arrays. Similarly, any d dimensional array is an array of $(d-1)$ dimensional arrays.

Generally, when we talk about multi-dimensional arrays, we generally talk of two-dimensional arrays. Since more than two-dimensional is rarely needed.

In C language a two Dimensional Array is implemented only by row major order.

Declaration of Two-Dimensional Array

The declaration of two-dimensional arrays is as follows:

```
data_type array_name [row_size][column_size];
```

Example:

```
int m[2][3];
```

	[0]	[1]	[2]
[0]	m[0][0]	m[0][1]	m[0][2]
[1]	m[1][0]	m[1][1]	m[1][2]

Initialization of Two-Dimensional Arrays

```
int matrix[2][3] = {1, 1, 1, 2, 2, 2};
```

Initializes all the elements of first row to 1 and the second row to 2. The individual array elements are as follow:

```
matrix [0][0] = 1 matrix [0][1] = 1 matrix[0][2] = 1
```

```
matrix [1][0] = 2 matrix [1][1] = 2 matrix[1][2] = 2
```

The initialization is done row by row. The above statement can be equivalently written as

```
int matrix[2][3] = {{1, 1, 1}, {2, 2, 2}};
```

Reading and Writing Two-Dimensional Array

```
int m[3][3];  
/* Reading */  
for(i = 0; i < 3; i++){  
    for(j = 0; j < 3; j++){  
        scanf("%d", &m[i][j]);  
    }  
}
```

```
/* Writing */  
for(i = 0; i < 3; i++){  
    for(j = 0; j < 3; j++){  
        printf("%d ", m[i][j]);  
        printf("\n ");  
    }  
}
```

Pointer

Pointer

The pointers are one of the most important and powerful data structure.

A pointer is a variable that is used to store an address or location of another variable.

Pointer declaration :

```
data-type *pointer-variable-name;
```

Examples:

```
int *p;  
char *q;  
float *r;
```

Example

```
#include<stdio.h>
main()
{
char c = 'C';
int a = 50;
float f = 3.45;
printf("Address of c = %u, Value of c = %c\n", &c, c);
printf("Address of a = %u, Value of a = %d\n", &a, a);
printf("Address of f = %u, Value of f = %f\n", &f, f);
}
Output:
Address of c = 65529, Value of c = C
Address of a = 65522, Value of a = 50
Address of f = 65518, Value of f = 3.450000
```

Address	Value	Variable name
	:	
65529	67	c
65522	50	a
65518	3.45	f
	:	
Memory		

Variables in Memory

Indirect Operator

The indirection operator (*) returns the value of the memory address which is stored in its operand.

Address of operator (&) returns the address of a variable and indirection operator (*) return the value of that address.

Example

```
#include<stdio.h>
main()
{
int a = 50, b;
int *p;
p = &a;
b = *p;
printf("Value of a = %d\n", a);
printf("Address of a = %u\n", &a);
printf("Value of p = %u\n", p);
printf("Value of *p = %d\n", *p);
printf("Value of b = %d\n", b);
printf("Address of b = %u\n", &b);
printf("Address of p = %u\n", &p);
}
```

Output:

```
Value of a = 50
Address of a = 65524
Value of p = 65524
Value of *p = 50
Value of b = 50
Address of b = 65522
Address of p = 65520
```

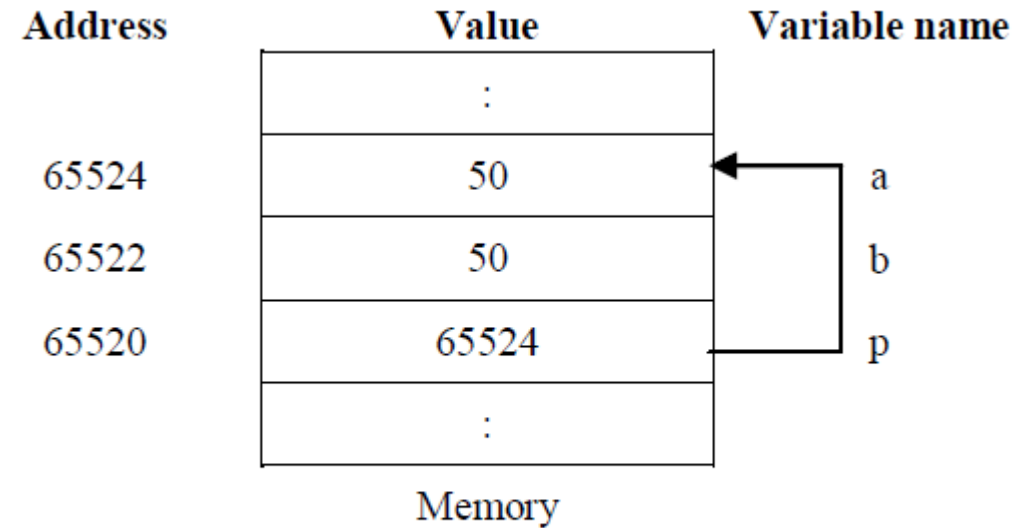


Figure 4.2: Variables in memory

Functions with Pointer

```
#include<stdio.h>
void swapi(int a, int b);
void swapf(float *a, float *b);
main()
{
    int i = 10, j = 20;
    float x = 12.3, y = 53.4;
    swapi(i, j);
    printf("i = %d, j = %d\n", i, j);
    swapf(&x, &y);
    printf("x = %f, y = %f", x, y);

    return 0;
}
```

```
/* Function to interchange two integer type variables*/
void swapi(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

```
/* Function to interchange two floating-point type variables*/
void swapf(float *a, float *b)
{
    float temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

The output..

```
i = 10, j = 20
x = 53.400002, y = 12.300000
Process returned 0 (0x0)   execution time : 0.447 s
Press any key to continue.
```

Call By Value

```
#include <stdio.h>

void penjumlahan (int bil1, int bil2, int jumlah){
    jumlah = bil1 + bil2;
    printf("*** Pemanggilan di dalam prosedur ***\n");
    printf("Jumlah %d ditambah %d = %d\n", bil1, bil2, jumlah); //jumlah = 3
    printf("*****\n\n");
}

int main(){
    int b1,b2,jum;
    b1 = 1;
    b2 = 2;
    jum = 0;

    penjumlahan (b1,b2,jum);

    printf("*** Pemanggilan di dalam main program ***\n");
    printf("Jumlah %d ditambah %d = %d\n", b1, b2, jum); //jumlah = 0
    printf("*****\n");

    getchar();
    return 0;
}
```

Call By Reference

```
#include <stdio.h>

void penjumlahan (int bil1, int bil2, int *jumlah){
    *jumlah = bil1 + bil2;
    printf("*** Pemanggilan di dalam prosedur ***\n");
    printf("Jumlah %d ditambah %d = %d\n", bil1, bil2, *jumlah); //jumlah = 3
    printf("*****\n\n");
}

int main(){
    int b1,b2,jum;
    b1 = 1;
    b2 = 2;
    jum = 0;

    penjumlahan (b1,b2,&jum);

    printf("*** Pemanggilan di dalam main program ***\n");
    printf("Jumlah %d ditambah %d = %d\n", b1, b2, jum); //jumlah = 3
    printf("*****\n");

    getchar();
    return 0;
}
```

Pointers and Arrays

Arrays are nothing but pointers. Hence, there is no index overflow or underflow checking. Arrays are also called as a static pointer. Since the name of an array is a constant pointer.

Example:

```
int a[5] = {10, 20, 30, 40, 50};
```

```
int i;
```

```
a[5] = 5; /* overflow */
```

```
a[-5] = -5; /* underflow */
```

No index overflow or underflow error message will be given by the compiler and index overflow or underflow checking does not take place at runtime.

Pointers and Arrays (2)

An array element **a[i]** can be written as ***(a+i)** in pointer representation and **a+i** signify the address of that element.

Example:

```
int a[5] = {10, 20, 30, 40, 50};
```

```
int i;
```

```
for(i=0; i<5;i++)
```

```
printf("%d", *(a+i));
```

Exercises

1. Write a program to compute sum of all elements stored in an array.
2. Write a program to sort an array in ascending order.

References

Pal, Debdutta & Suman Halder. 2018. Data Structures and Algorithms with C. Oxford, U.K. : Alpha Science International Ltd.

Weiss, M.A. 2014. Data Structures and Algorithm Analysis in C++, 4th Edition, International Edition. Great Britain: Pearson Education Limited.